

Bonus assignment :

Ganesh Gundekarla

11700551

Coding on Ranking of web results:

Explanation :

First we start off by declaring some of the constants to work with.

1. Here , the total number of pages are set to be 34279
2. The only number of top pages to be extract is 20.
3. Here , the minimum value of a random page weights should not be below the -80000
4. The maximum weight buffer is set to 50000.

a. Generating random weights for the page.

For this , we would be using a random uniform function which would be able to generate an array of random weights which will be under the range of (minimum_weight_buffer, max weight_buffer) .

b. List conversion.

Here, the numpy array the above one named 'page_weights' is then converted to a regular python list which is named as 'weights_of_pages_list'. Here , this conversion is an important step as the sorting algo named heapq works with lists data type in python only .

c. Time calculation

Here, we set the starting time to the current time before running the main program and the ending_time is set to the current time which after the sorting and the process of extraction is being done . We then calculate the elapsed time of the process by subtracting the 'starting time ' and the 'ending_time' .

d. Heap sort .

1. Here , the weights are then converted to heap structure using the heapify which effectively creates an heap (min) .
2. Then , the top pages are being extracted by using the heappop which is done and run for 'no_of_pages' times . this is done in ascending order .

3. After the sorting is done , the list is then reversed to get the weights in order to get those in descending order .

e. Top elements .

1. Here , the first k elements from the sorted one are then being sorted and then stored in the top k weights .

Output: top 20 weights , elapsed time .

Top 20 weights:

49997.13373149396

49995.32241053831

49995.062254369695

49994.35965850424

49993.36612793918

49991.0491124461

49989.43677805284

49981.231458817725

49981.185912264875

49975.821702424844

49975.115155205494

49974.20670152975

49974.188374307516

49973.244820803055

49972.364056021936

49963.92463963902

49963.31293155842

49962.08924074049

49955.99719125952

49955.26007390044

Elapsed time: 0.060698 seconds

Code :

```
def main():
```

```
    # Constants
```

```
    No_of_pages = 34279 # Total number of pages
```

```
    K = 20    # Number of top pages to extract
```

```
    min_weight_buffer = -80000
```

```
    max_weight_buffer = 50000
```

```

# Generate random page weights
np.random.seed(0) # Seed for reproducibility
page_weights = np.random.uniform(min_weight_buffer, max_weight_buffer,
No_of_pages)

# Convert numpy array to list for compatibility with heapq
weights_of_pages_list = list(page_weights)

# Start time measurement
starting_time = time.time()

# Sort the page weights using heap sort
heapq.heapify(weights_of_pages_list)
sorted_weights_accordingly = [heapq.heappop(weights_of_pages_list) for _
in range(No_of_pages)]
sorted_weights_accordingly.reverse() # Get the weights in descending order

# Extract the top K elements
top_k_weights = sorted_weights_accordingly[:K]

# End time measurement
ending_time = time.time()
elapsed_time = ending_time - starting_time

# Output the results
print("Top 20 weights:")
for weight in top_k_weights:
    print(weight)
print(f"Elapsed time: {elapsed_time:.6f} seconds")

if __name__ == "__main__":
    main()

```

```
+ Code + Text ✓ RA
[1] import numpy as np
import heapq
import time

def main():
    # Constants
    No_of_pages = 34279 # Total number of pages
    K = 20 # Number of top pages to extract
    min_weight_buffer = -80000
    max_weight_buffer = 50000

    # Generate random page weights
    np.random.seed(0) # Seed for reproducibility
    page_weights = np.random.uniform(min_weight_buffer, max_weight_buffer, No_of_pages)

    # Convert numpy array to list for compatibility with heapq
    weights_of_pages_list = list(page_weights)

    # Start time measurement
    starting_time = time.time()

    # Sort the page weights using heap sort
    heapq.heapify(weights_of_pages_list)
    sorted_weights_accordingly = [heapq.heappop(weights_of_pages_list) for _ in range(No_of_pages)]
    sorted_weights_accordingly.reverse() # Get the weights in descending order

    # Extract the top K elements
    top_k_weights = sorted_weights_accordingly[:K]

    # End time measurement
    ending_time = time.time()
    elapsed_time = ending_time - starting_time

    # Output the results
    print("Top 20 weights:")
    for weight in top_k_weights:
        print(weight)
    print(f"Elapsed time: {elapsed_time:.6f} seconds")

if __name__ == "__main__":
    main()

Top 20 weights:
49997.13373149396
49995.32241053831
49995.062254360695
49994.35965850424
49993.36612793918
49991.0491124461
49989.43677805284
49981.231458817725
49981.185912264875
49975.821702424844
49975.115155205494
49974.20670152075
49974.188374307516
49973.244820803055
49972.364056021936
49963.92463963902
49963.31293155842
49962.08924074049
49955.99719125952
49955.26007390044
Elapsed time: 0.060698 seconds
```

Time complexity calculation :

total time complexity= $O(n+k\log(n))$.

number of iterations = No_of_pages.

calculate time complexity

```
def main():

    N = 34279      # ---- time complexity O(n)
    K = 20
    min_weight = -80000
    max_weight = 50000

    np.random.seed(0)
    page_weights = np.random.uniform(min_weight, max_weight, N) # time complexity O(n)
    page_weights_list = list(page_weights) # time complexity is O(n) .

    # Start time measurement
    start_time = time.time()

    # Sort the page weights using heap sort
    heapq.heapify(page_weights_list) # heapify operation on list weights is O(n) .
    sorted_weights = [heapq.heappop(page_weights_list) for _ in range(N)]
    sorted_weights.reverse() # Get the weights in descending order

    # Extract the top K elements
    top_k_weights = sorted_weights[:K]

    # End time measurement
    end_time = time.time()
    elapsed_time = end_time - start_time

    # Output the results
    print("Top 20 weights:")
    for weight in top_k_weights: # extracting the top k elements
        print(weight) # here , k iterations , each iteration is O(logn) time taken
    print(f"Elapsed time: {elapsed_time:.6f} seconds") # total time = O(klog(n)) seconds .

if __name__ == "__main__":
    main()

# total time complexity= O(n+klog(n)).
```

Screenshot

[10]